

IFAPME - X75-2

Développeur web front end 2022-2023

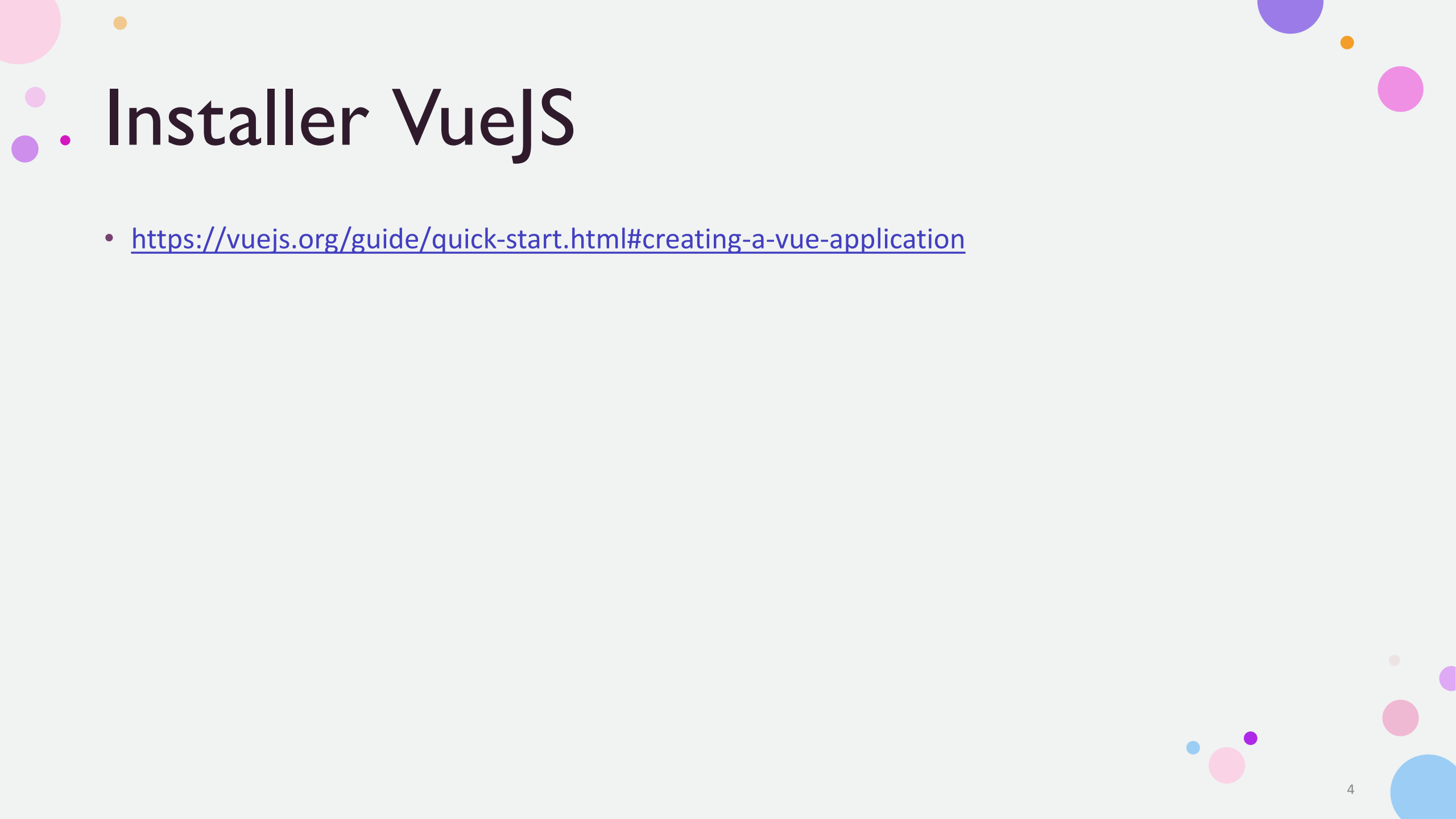
ADW - VueJS

VueJS



Quand utiliser VueJS ?

- Enhancing static HTML without a build step
- Embedding as Web Components on any page
- **Single-Page Application (SPA)**
- Fullstack / Server-Side Rendering (SSR)
- Jamstack / Static Site Generation (SSG)
- Targeting desktop, mobile, WebGL, and even the terminal



Installer VueJS

- <https://vuejs.org/guide/quick-start.html#creating-a-vue-application>

Single-File Components (SFC)

- Les composants permettent d'encapsuler une logique de fonctionnement dans un seul fichier (.vue)
- On y retrouve 3 parties :
 - `<script>` : La logique du composant, comment doit-il réagir et quelles sont les différentes interactions possibles
 - `<template>` : La partie rendu, c'est ce qui sera affiché dans le navigateur en fonction de la logique à employer dans `<script>`
 - `<style>` : On peut styliser un composant directement dans le même fichier. Avec l'attribut **scoped** on peut définir que le CSS ne pourra fonctionner qu'avec `<template>` dans ce fichier.

```
<script>
export default {
  data() {
    return {
      count: 0
    }
  }
}
</script>
```

```
<template>
  <button @click="count++">Count is: {{ count }}</button>
</template>
```

```
<style scoped>
button {
  font-weight: bold;
}
</style>
```

Afficher une variable

- On entoure la variable que l'on souhaite afficher par {{ ... }}
- Ou bien en utilisant l'attribut v-html, on lui passe la variable en tant qu'argument
- `<p>Using text interpolation: {{ ma_variable }}</p>`
`<p>Using v-html directive: <span v-html="ma_variable"></span></p>`

Fondamentaux de la réactivité

- Toutes les variables définies dans la partie *data()* sont sujettes à être utilisé dans la partie rendu.
- Ces variables peuvent être modifiées à tout moment et le rendu sera directement changé dans le navigateur
- Par exemple :
 - On affiche une variable *text* dans le code (Ex: *hello world*)
 - Le navigateur affiche le contenu de *tekst*
 - Si (par un évènement quand on clique sur un bouton, par ex.), on modifie le contenu de cette variable
 - Cela sera directement mis à jour dans le rendu du navigateur

Ajouter des méthodes à notre composant

- On peut créer une section *methods* dans notre logique composant. Ces méthodes pourraient être appelées à différents moments :
 - Par exemple, lorsqu'on clique sur un bouton
 - Pour scinder du code proprement, réutilisable

```
• export default {  
  data() {  
    return {  
      text: 'Hello world!'  
    }  
  },  
  
  methods: {  
    ma_fonction() {  
      // ...  
    }  
  }  
}
```


Binding d'attributs

- On peut dynamiquement ajouter un attribut à une balise HTML, c'est-à-dire passer une variable en tant que paramètre
- `<div v-bind:id="dynamicId"></div>`
`<div :id="dynamicId"></div>`
- On peut également directement passer un booléen à certains attributs. Dans l'exemple ci-dessous, si la variable *isButtonDisabled* == *true* alors l'attribut *disabled* sera ajouté à la balise *button*
- `<button :disabled="isButtonDisabled">Button</button>`

Binding d'attributs

- On peut également directement binder un objet contenant différents attributs pour gagner du temps et de la lisibilité

- ```
data() {
 return {
 objectOfAttrs: {
 id: 'container',
 class: 'wrapper'
 }
 }
}
```

- ```
<div v-bind="objectOfAttrs"></div>
```

Les computed properties

- Au lieu d'utiliser des méthodes pour afficher un résultat dans le code HTML, on préférera utiliser les *computed properties*
- Elles ont l'avantage de n'être mise à jour que lorsque la partie `data()` est mise à jour (à la différence à une méthode qui, affichée dans l'HTML, sera exécutée à chaque cycle de l'application)
- `computed: {`
 - `// a computed getter`
 - `publishedBooksMessage() {`
 - `// `this` points to the data() of the component instance`
 - `return this.author.books.length > 0 ? 'Yes' : 'No'`
 - `}`

Class binding

- Le class binding permet, comme pour les attributs, d'affecter des classes dynamiquement selon une condition ou une variable
- ```
data() {
 return {
 isActive: true,
 hasError: false
 }
},
```
- ```
<div class="static" :class="{ active: isActive, 'text-danger': hasError }" ></div>
```

Rendu conditionnel

- En VueJS, il est possible de choisir d'afficher une section de code HTML, ou pas, selon une ou plusieurs conditions
- `<h1 v-if="isMorning">Bonjour</h1>`
`<h1 v-else-if="isEvening">Bonsoir</h1>`
`<h1 v-else>Hello</h1>`
- Il existe également *v-show* qui a la particularité de masquer un élément, mais seulement par une propriété CSS. C'est-à-dire que l'élément sera toujours dans le DOM de la page et qui pourrait être manipulé même s'il est masqué

Rendu de listes et tableaux

- Lorsqu'on manipule des tableaux (array), il est évidemment possible de pouvoir itérer dans ce tableau pour afficher son contenu (dans une liste par exemple)
- ```
data() {
 return {
 items: [{ message: 'Foo' }, { message: 'Bar' }]
 }
},
```
- ```
<li v-for="item in items">  
  {{ item.message }}  
</li>
```
- Dès que possible, on passera un attribut `:key=""` qui permettra d'identifier plus facilement un élément dans une liste (notamment lorsqu'on passera un composants dans la liste)
- ```
<template v-for="todo in todos" :key="todo.name">
 {{ todo.name }}
</template>
```

# Les évènements

- Les évènements utilisés sont exactement les même que l'on utiliserait en javascript avec `.addEventListener(...)`
- `data() { return { count: 0 } }`
- `<button @click="count++">Add 1</button>`  
`<p>Count is: {{ count }}</p>`
- On peut également utiliser des directives intéressantes, pour effectuer des actions spécifiques
- `<!-- the submit event will no longer reload the page -->`  
`<form @submit.prevent="onSubmit"></form>`

# Les évènements

- On peut bien entendu appeler directement une méthode lorsqu'un évènement se produit
- ```
methods: {  
  say(message) {  
    alert(message)  
  }  
}
```
- ```
<button @click="say('hello')">Say hello</button>
<button @click="say('bye')">Say bye</button>
```



# Form Input Binding

- Il est tout à fait possible de lier des champs d'un formulaire à la partie *data()* d'un composant. Cela permet notamment de pouvoir interagir avec ces données plus tard (lors de la soumission du formulaire par exemple)
- ```
<p>Message is: {{ message }}</p>  
<input v-model="message" placeholder="edit me" />
```
- ```
<textarea v-model="text"></textarea>
```
- ```
<input type="checkbox" id="checkbox" v-model="checked" />  
<label for="checkbox">{{ checked }}</label>
```
- Pour des checkbox, il est tout à fait possible d'utiliser un tableau dans *data()* qui contiendra toutes les valeurs, plutôt que de créer une variable par checkbox.
- ```
<div>Checked names: {{ checkedNames }}</div>

<input type="checkbox" id="jack" value="Jack" v-model="checkedNames">
<label for="jack">Jack</label>

<input type="checkbox" id="john" value="John" v-model="checkedNames">
<label for="john">John</label>

<input type="checkbox" id="mike" value="Mike" v-model="checkedNames">
<label for="mike">Mike</label>
```

# Lifecycle Hooks

- Il existe une multitude d'évènements où l'on peut effectuer des actions spécifiques pendant la durée de vie d'un composant
- Par exemple, *mounted()* sera exécuté après le premier rendu du composant et lorsque le DOM sera initialisé
- ```
export default {  
  mounted() {  
    console.log('the component is now mounted.')  
  }  
}
```
- <https://vuejs.org/guide/essentials/lifecycle.html#lifecycle-diagram>

Les Watchers

- Les watchers fonctionnent un peu comme les computed properties
- La différence est qu'on les utilise plutôt dans un contexte de modifier du DOM ou bien d'exécuter du code asynchrone

```
• data() {  
  return {  
    question: "",  
  }  
},  
watch: {  
  // whenever question changes, this function will run  
  question(newQuestion, oldQuestion) {  
    if (newQuestion.includes('?')) {  
      this.getAnswer()  
    }  
  }  
}  
},
```

Template Refs

- Les *refs* permettent directement d'accéder aux éléments du DOM sans devoir passer par les opérations traditionnelles du Javascript *document.getElementById()*, ...
- VueJS crée son propre DOM en mémoire, ce qui lui permet d'effectuer des manipulations un peu différentes du DOM classique
- On utilise ainsi la propriété *ref=""* plutôt que l'accès via l'ID ou les class d'un élément du DOM
- `<input ref="input">`
- ```
mounted() {
 this.$refs.input.focus()
}
```
-

# Création de nouveaux composants

- L'intérêt de VueJS, c'est de découper son code en plusieurs composants
- Chaque composant doit garder sa propre logique métier et doit conserver une approche réutilisable
  - Un composant pour la navbar
    - Un composant pour chaque lien de la navbar
  - Un composant pour une barre de recherche
  - Un composant pour un bouton
  - ...
- Pour chaque composant, on créera son propre fichier .vue

# Utiliser un composant dans un autre

- ```
import ButtonCounter from './ButtonCounter.vue'
export default {
  components: {
    ButtonCounter
  }
}
```
- ```
<template>
 <h1>Here is a child component!</h1>
 <ButtonCounter />
</template>
```

# Passer des données dans un composants (les props)

- On peut passer des données / arguments dans un composant pour faire réagir sa logique métier en fonction des besoins

- ```
<script>
export default {
  props: ['title']
}
</script>

<template>
  <h4>{{ title }}</h4>
</template>
```

- ```
<BlogPost title="My journey with Vue" />
<BlogPost title="Blogging with Vue" />
<BlogPost title="Why Vue is so fun" />
```

# Réagir à un évènements d'un composant

- L'idée est de faire réagir le parent d'un composant lorsqu'un évènement se produit au sein de ce même composant
- Par exemple, un composant contient un bouton, et on veut que le parent effectue une action
- `// Composant parent`  
`<TodoListForm @form-submit="mafonction()"/>`
- `// Composant enfant`  
`<button @click="$emit('form-submit') »>Envoyer</button>`
- On peut bien sûr passer des données dans cet appel de fonction
- `<button @click="$emit('increaseBy', 1)">`  
Increase by 1  
`</button>`
- `<MyButton @increase-by="increaseCount" />`
- `methods: {`  
  `increaseCount(n) { this.count += n }`  
  `}`



# Slot

- Comme pour une balise HTML, on peut passer aussi du contenu à un composant de cette manière
- `<AlertBox>Something bad happened.</AlertBox>`
- ```
<template>
  <div class="alert-box">
    <strong>This is an Error for Demo Purposes</strong>
    <slot />
  </div>
</template>
```

Exercice 01 – Une barre de recherche

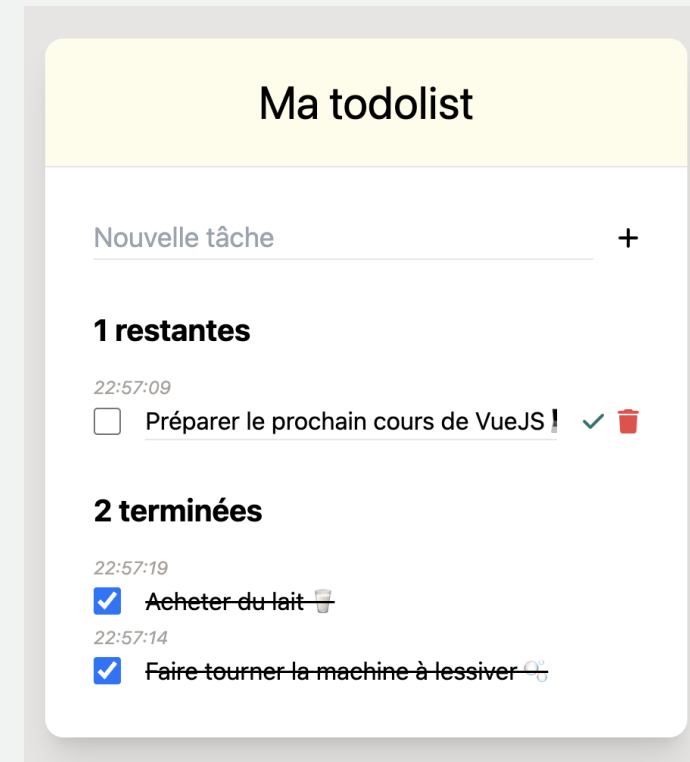
- Créer une application VueJS qui permettra d'effectuer une recherche dans un Input et d'afficher les résultats dans une liste
- Il doit y avoir 4 composants
 - Le composant principal App
 - La barre de recherche SearchBar
 - Le composant de résultat global SearchBarResultList
 - Le composant de résultat SearchBarResultItem
- Le composant principal App doit contenir une liste d'éléments à rechercher dans un tableau (Par exemple une liste de villes)
- Les résultats doivent se mettre à jour lorsque qu'un utilisateur modifie l'Input de recherche (Voir les événements dans la documentation de VueJS)
- On utilisera la propriété *slot* dans *SearchBarResultItem* pour afficher un résultat d'une réponse

Exercice 02 – Une application qui affiche la météo d'une ville

- Grâce à l'API et axios <https://www.weatherapi.com/> qui est disponible gratuitement, créer une application qui permet d'entrer le nom d'une ville et d'en connaître la météo
- Il faut afficher une image d'un thermomètre qui varie selon la météo
- Exemple de query :
`http://api.weatherapi.com/v1/current.json?key=621e595263f642eaac6164645233001&q=VILLE&aqi=no`
- Il doit y avoir 3 composants
 - Le composant principal App
 - Un composant *CityInput* qui permet d'entrer le nom de la ville
 - Un composant *WeatherResult* qui permet d'afficher la réponse de l'API
- La recherche de ville ne doit s'effectuer que lorsqu'on valide le formulaire et si l'input fait au moins 3 caractères (Afficher un message d'erreur si ce n'est pas le cas)
- Afficher l'icône de météo directement retournée dans la réponse de l'API de weatherapi
- Si l'API nous renvoie une erreur, il faut afficher cette erreur dans WeatherResult

Exercice 03 – Une todolist

- Créer une todolist respectant les fonctionnalités démontrées dans ces deux images



Exercices

- Créer une liste de tâches à faire en utilisant des composants VueJS.
- Afficher une liste de produits avec des images et des descriptions en utilisant un tableau et une boucle.
- Créer un formulaire d'inscription avec des champs de saisie de données.
- ~~• Créer une barre de recherche qui filtre les données d'une liste en temps réel.~~
- Créer un mini-jeu de devinette avec des messages de succès/échec en utilisant des conditions.
- ~~• Créer une application de météo qui affiche la température et les conditions météorologiques pour une ville spécifiée.~~
- Implémenter la navigation entre les pages d'une application en utilisant Vue Router.